

Documentación Técnica — Sistema de Gestión de Biblioteca

1. Introducción

Una pequeña biblioteca necesita informatizar el registro y consulta de sus libros. La información se almacenaba de manera manual, dificultando la búsqueda, actualización y administración de los ejemplares disponibles.

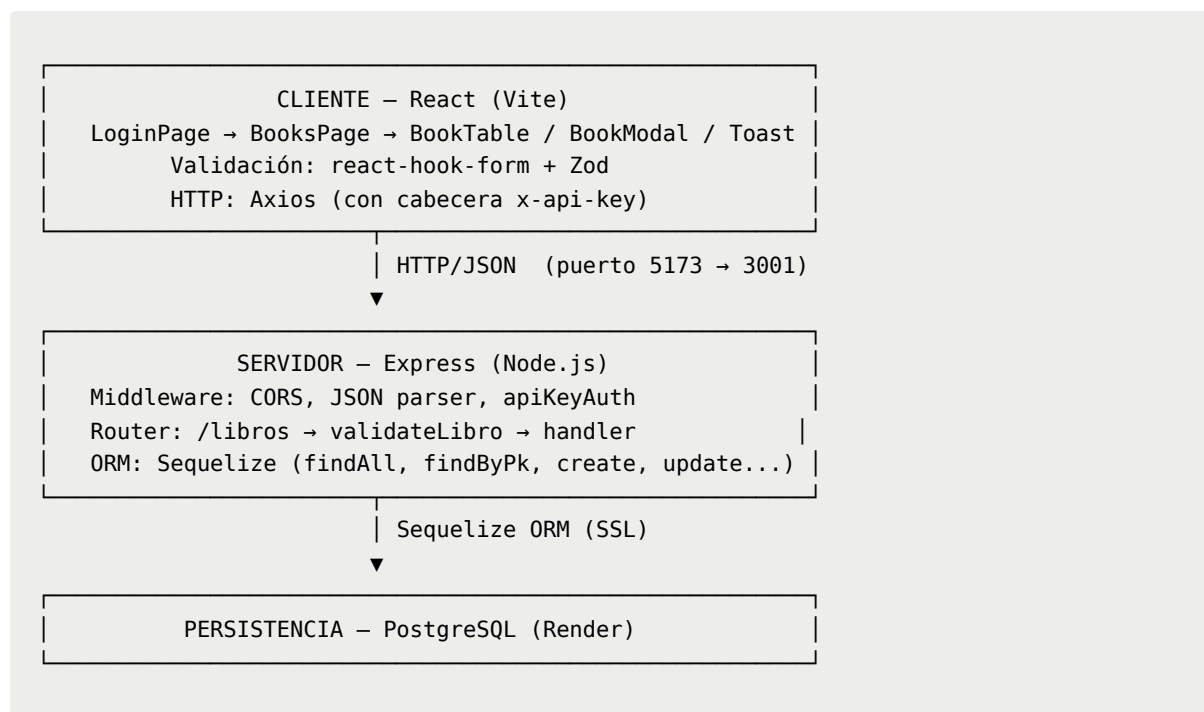
El presente trabajo implementa una aplicación web full-stack que resuelve este problema mediante una arquitectura cliente-servidor. El backend expone una API REST desarrollada con Node.js y Express, que gestiona el catálogo de libros y persiste los datos en una base de datos PostgreSQL a través del ORM Sequelize. El frontend, desarrollado con React y TypeScript, consume dicha API y ofrece una interfaz gráfica intuitiva para que los operadores de la biblioteca puedan administrar el catálogo sin conocimientos técnicos.

Se implementaron, además de los requisitos mínimos, las siguientes funcionalidades extra:

- Búsqueda por título en tiempo real.
- Filtro por disponibilidad.
- Persistencia en base de datos PostgreSQL (Render) mediante Sequelize ORM.
- Autenticación por API Key en el backend.
- Pantalla de login con validación en el frontend.

2. Arquitectura de la Solución

La aplicación sigue una arquitectura cliente-servidor de dos capas desacopladas:



Tecnologías utilizadas

Capa	Tecnología	Versión
Backend	Node.js + Express	4.18.x
Backend	Sequelize (ORM)	6.x
Backend	pg + pg-hstore	8.x
Backend	dotenv	17.x
Backend	cors	2.8.x
Frontend	React	18.2.x
Frontend	TypeScript	5.2.x
Frontend	Vite	5.0.x
Frontend	Axios	1.6.x
Frontend	react-hook-form	7.48.x
Frontend	Zod	3.22.x

Estructura de carpetas

```
libreria-express-react/  
├── backend/  
│   ├── src/  
│   │   ├── index.js           # Punto de entrada, conexión DB y Express  
│   │   └── config/
```

└─ db.js	# Instancia Sequelize (DATABASE_URL)
└─ models/	
└─ Libro.js	# Modelo Sequelize de la tabla libros
└─ routes/	
└─ libros.js	# Definición de los 5 endpoints
└─ middleware/	
└─ apiKeyAuth.js	# Autenticación por API Key
└─ validateLibro.js	# Validación de campos del libro
└─ frontend/	
└─ src/	
└─ App.tsx	# Router raíz (login / libros)
└─ pages/	
└─ LoginPage.tsx	# Pantalla de inicio de sesión
└─ BooksPage.tsx	# Pantalla principal de gestión
└─ components/	
└─ BookTable.tsx	# Tabla de libros
└─ BookModal.tsx	# Modal de alta y edición
└─ Toast.tsx	# Notificaciones temporales
└─ hooks/	
└─ useBooks.ts	# Lógica de estado y llamadas API
└─ services/	
└─ api.ts	# Instancia Axios y funciones HTTP
└─ schemas/	
└─ bookSchema.ts	# Esquema Zod para libro
└─ loginSchema.ts	# Esquema Zod para login
└─ types/	
└─ Book.ts	# Tipo TypeScript del libro

Flujo de autenticación

El frontend almacena el estado de sesión en `localStorage`. Al iniciar, `App.tsx` verifica la clave `isLoggedIn`; si no existe, muestra `LoginPage`. Las credenciales válidas son verificadas en el cliente. Todas las llamadas a la API incluyen la cabecera `x-api-key` cuyo valor proviene de la variable de entorno `VITE_API_KEY`.

3. Descripción de Endpoints

Base URL: `http://localhost:3001`

Todos los endpoints requieren la cabecera:

```
x-api-key: <valor configurado en .env>
```

3.1 — GET /libros

Retorna el listado completo de libros almacenados.

Ítem	Valor
Método	GET
Ruta	/libros
Autenticación	x-api-key
Body requerido	No
Respuesta exitosa	200 OK — array de libros

Ejemplo de respuesta:

```
[
  {
    "id": 1,
    "titulo": "El Principito",
    "autor": "Antoine de Saint-Exupéry",
    "anio": 1943,
    "disponible": true
  },
  {
    "id": 2,
    "titulo": "Cien años de soledad",
    "autor": "Gabriel García Márquez",
    "anio": 1967,
    "disponible": false
  }
]
```

3.2 — GET /libros/:id

Retorna un libro específico según su identificador numérico.

Ítem	Valor
Método	GET
Ruta	/libros/:id
Autenticación	x-api-key
Parámetro de ruta	id — número entero

Ítem	Valor
Respuesta exitosa	200 OK — objeto libro
Error	404 Not Found si no existe el id

Ejemplo: `GET /libros/1`

Respuesta:

```
{
  "id": 1,
  "titulo": "El Principito",
  "autor": "Antoine de Saint-Exupéry",
  "anio": 1943,
  "disponible": true
}
```

3.3 — POST /libros

Registra un nuevo libro en el catálogo. El `id` es asignado automáticamente por PostgreSQL como clave primaria autoincremental (SERIAL); no debe enviarse en el body.

Ítem	Valor
Método	POST
Ruta	<code>/libros</code>
Autenticación	x-api-key
Content-Type	application/json
Respuesta exitosa	201 Created — objeto libro creado
Error	400 Bad Request si falla la validación

Body requerido:

```
{
  "titulo": "Don Quijote de la Mancha",
  "autor": "Miguel de Cervantes",
  "anio": 1605,
  "disponible": true
}
```

Respuesta:

```
{
  "id": 5,
  "titulo": "Don Quijote de la Mancha",
  "autor": "Miguel de Cervantes",
  "anio": 1605,
  "disponible": true
}
```

3.4 — PUT /libros/:id

Actualiza todos los campos de un libro existente. El `id` no puede modificarse.

Ítem	Valor
Método	PUT
Ruta	<code>/libros/:id</code>
Autenticación	x-api-key
Content-Type	application/json
Respuesta exitosa	200 OK — objeto libro actualizado
Error 400	Si falla la validación de campos
Error 404	Si el libro no existe

Body requerido: misma estructura que POST.

3.5 — DELETE /libros/:id

Elimina un libro del catálogo. Retorna el objeto eliminado.

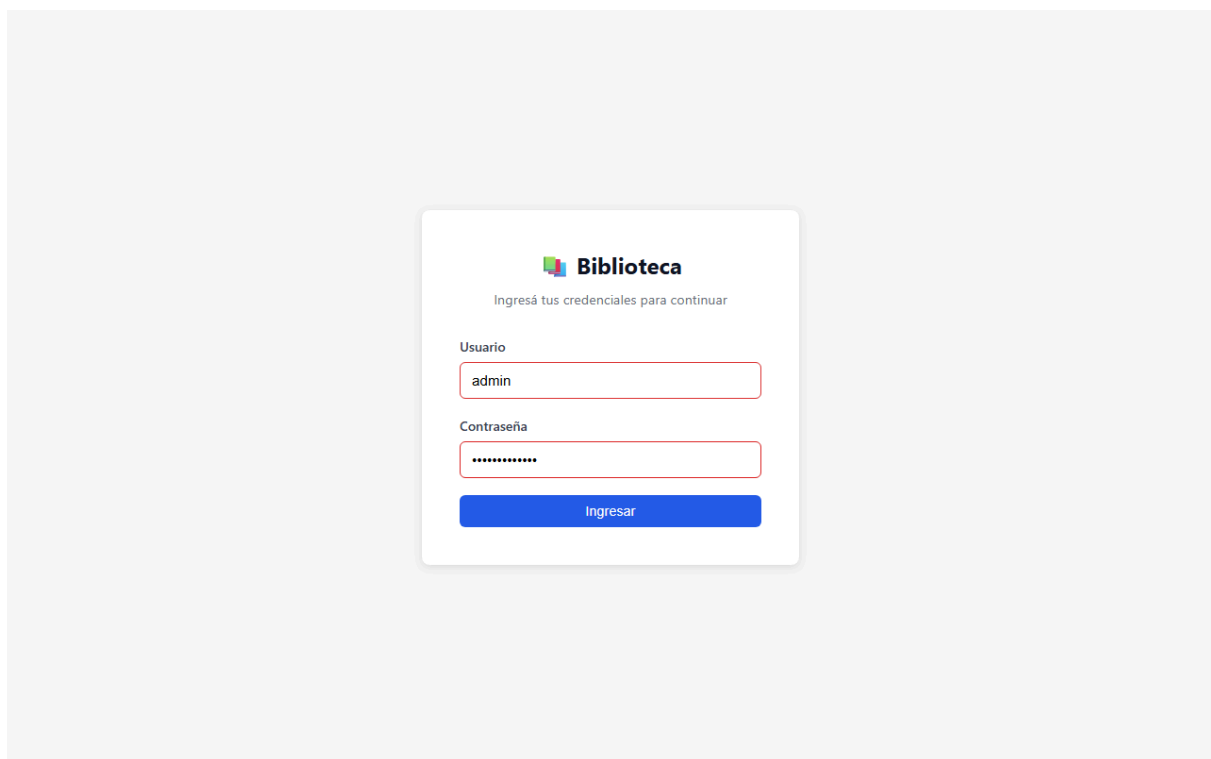
Ítem	Valor
Método	DELETE
Ruta	<code>/libros/:id</code>
Autenticación	x-api-key
Respuesta exitosa	200 OK — objeto libro eliminado
Error 404	Si el libro no existe

Resumen de respuestas de error

Código	Situación	Body ejemplo
400	Título vacío	<pre>{ "error": "El título es obligatorio" }</pre>
400	Autor vacío	<pre>{ "error": "El autor es obligatorio" }</pre>
400	Año inválido (< 1900 o > año actual)	<pre>{ "error": "El año debe ser un número entero entre 1900 y 2026" }</pre>
400	<code>disponible</code> no es boolean	<pre>{ "error": "El campo disponible debe ser verdadero o falso" }</pre>
401	API Key ausente o incorrecta	<pre>{ "error": "API key inválida o ausente" }</pre>
404	ID no encontrado	<pre>{ "error": "Libro no encontrado" }</pre>

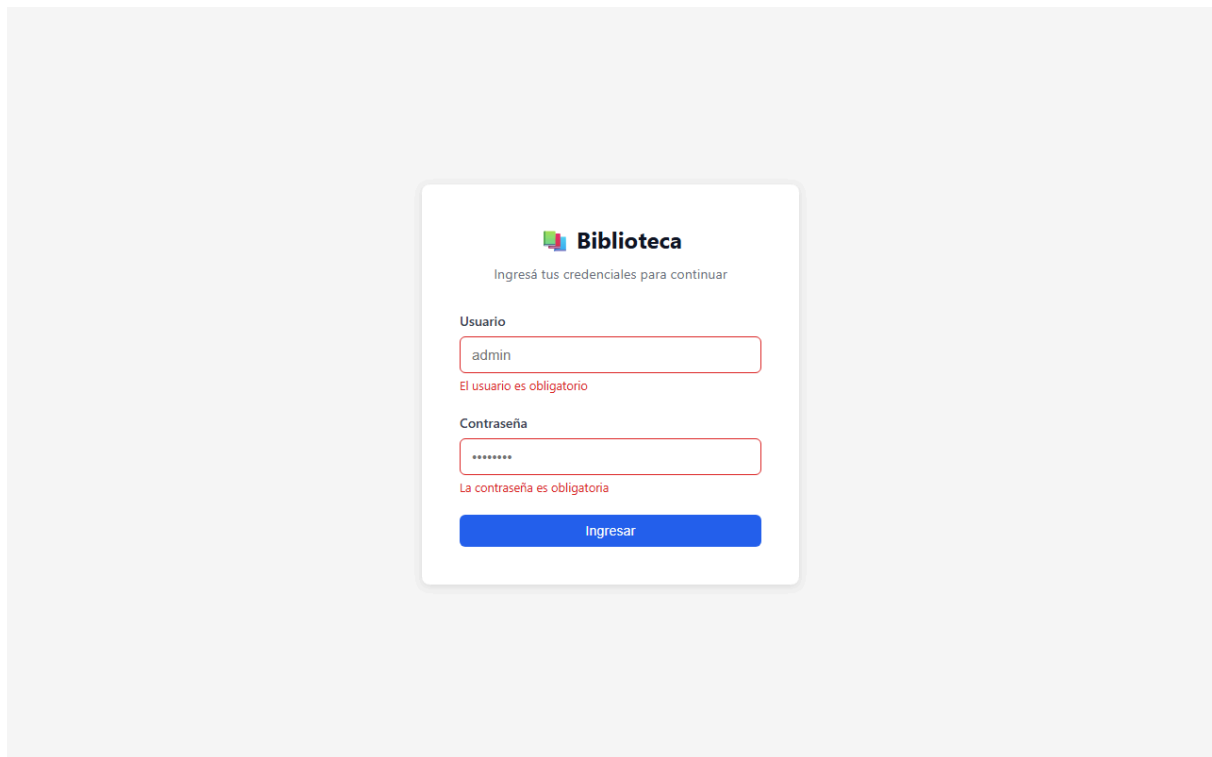
4. Capturas de Pantalla

4.1 Pantalla de Login



Pantalla de login con credenciales ingresadas

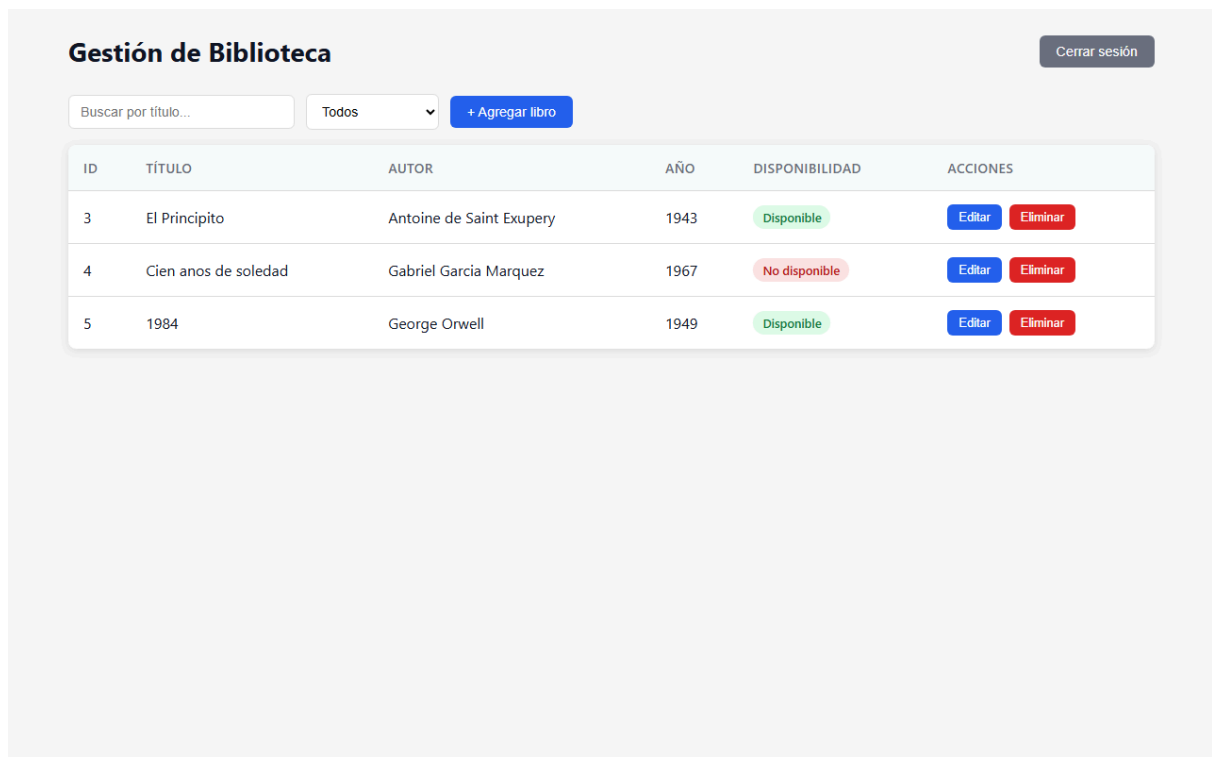
Acceso con usuario `admin` y contraseña `biblioteca123`.



Login con errores de validación

La validación con Zod muestra mensajes de error en tiempo real si los campos están vacíos.

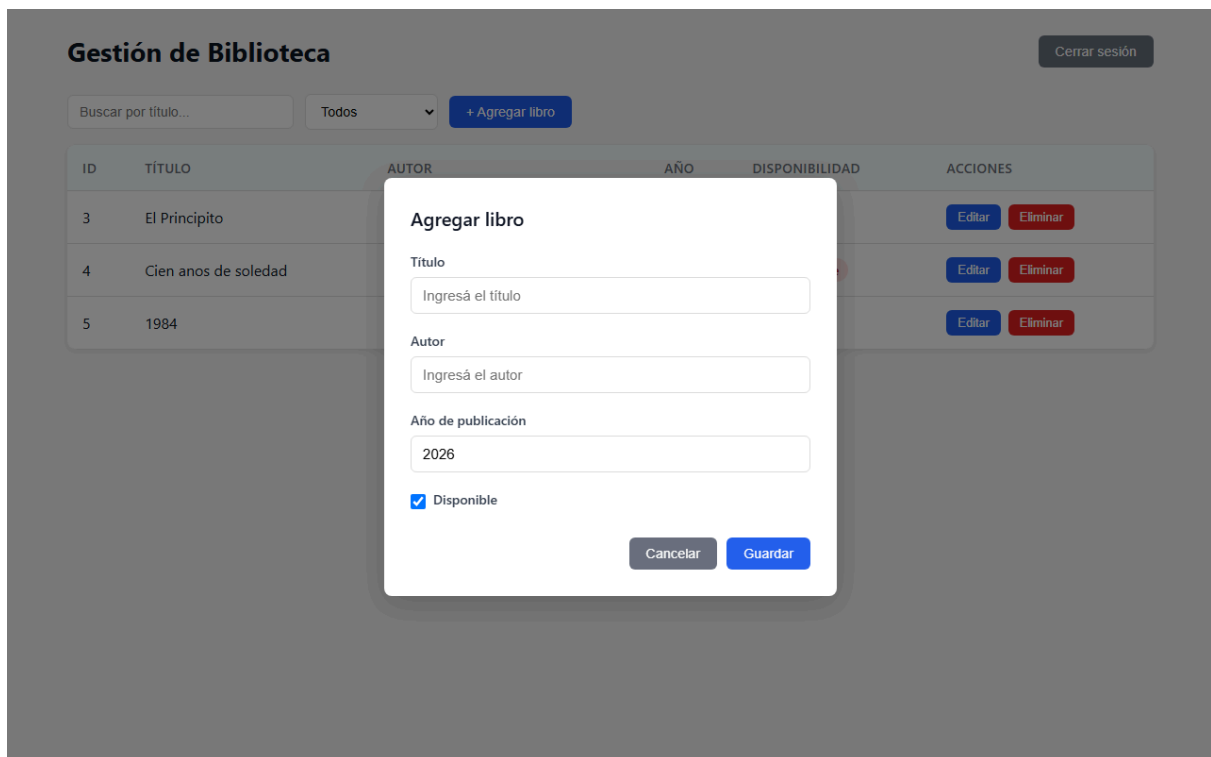
4.2 Listado de Libros



Pantalla principal con tabla de libros

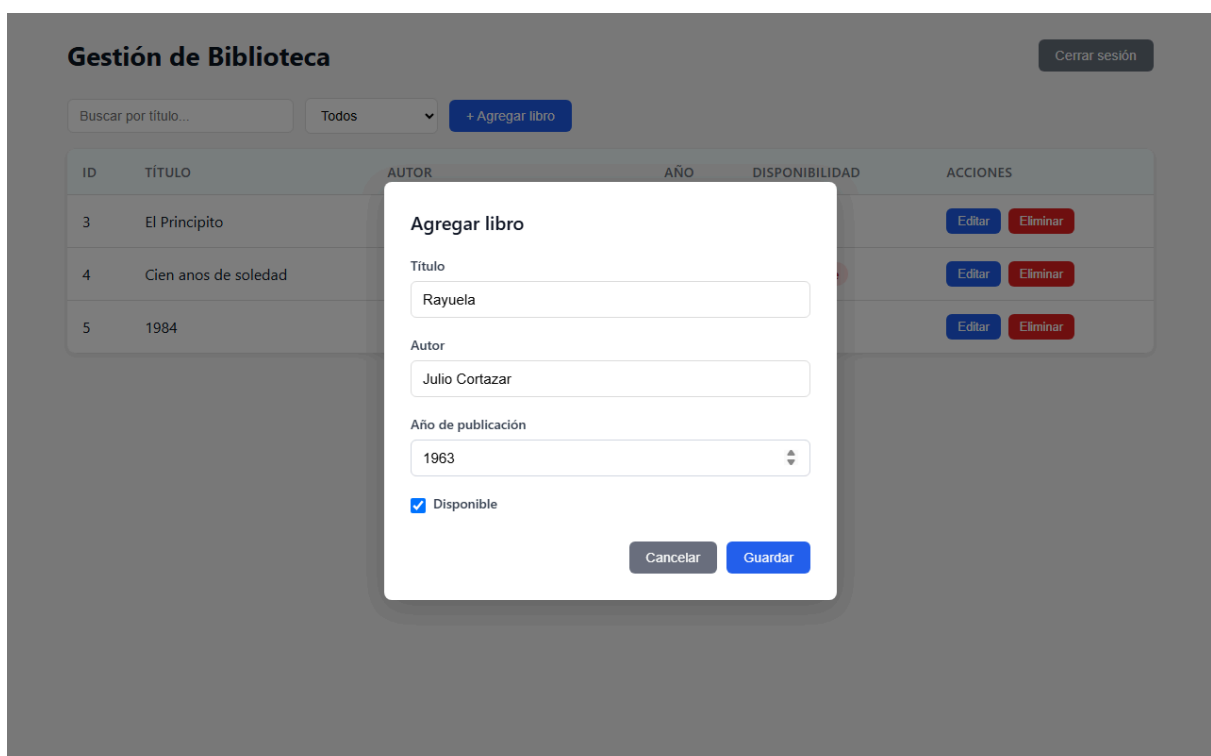
La tabla muestra ID, Título, Autor, Año, Disponibilidad (con badge verde/rojo) y los botones Editar y Eliminar por fila.

4.3 Alta de Libro (Modal de creación)



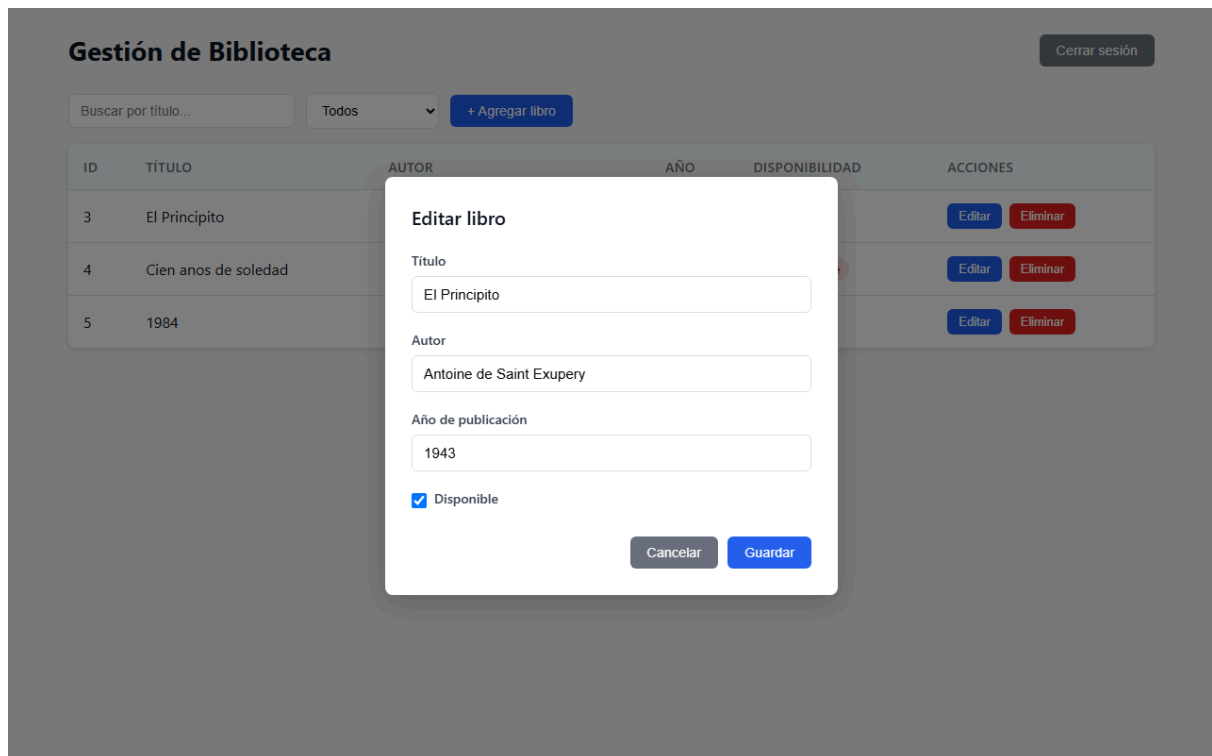
Modal de alta vacío

Al hacer clic en "+ Agregar libro" se abre el modal. Si se intenta guardar con campos inválidos, se muestran los mensajes de error del esquema Zod antes de enviar al servidor.



Modal de alta con datos completados

4.4 Modificación de Libro (Modal de edición)

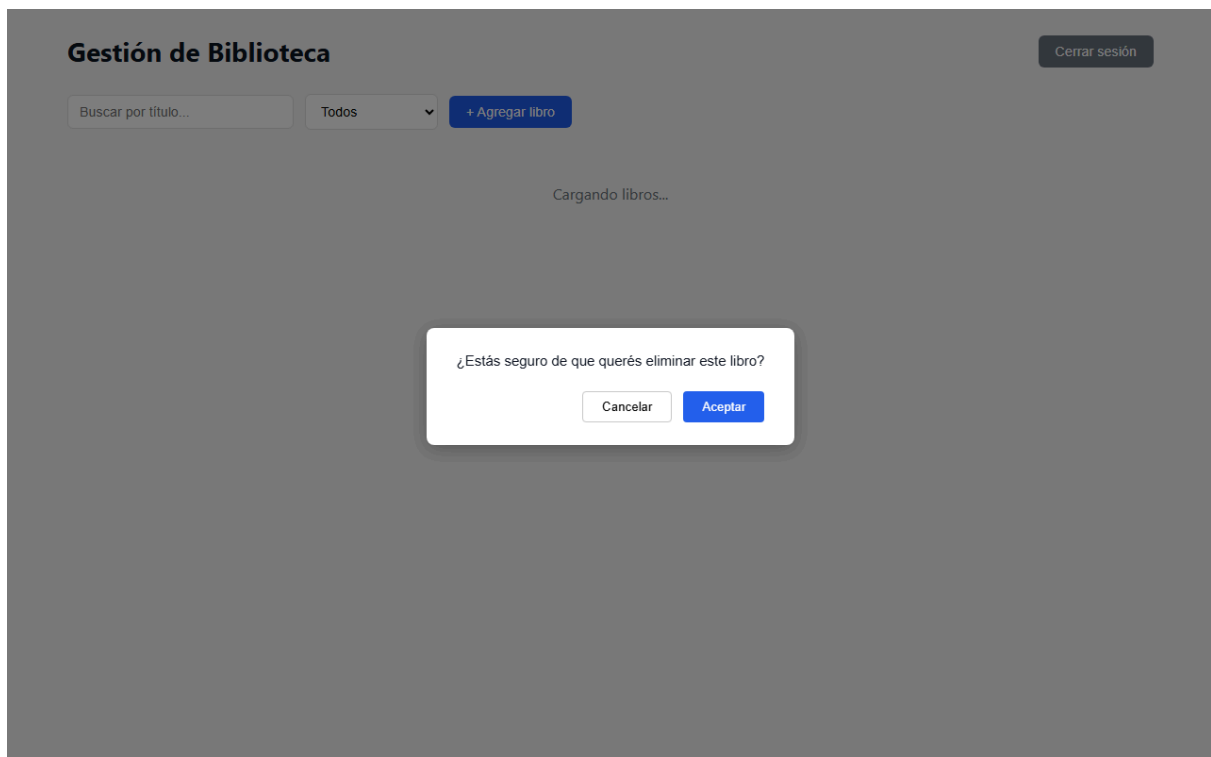


Modal de edición con datos prellenados

Al hacer clic en "Editar" el modal se precarga con los datos actuales del libro seleccionado.

Al guardar se realiza un PUT /libros/:id y se refresca la tabla.

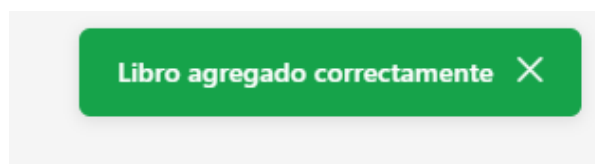
4.5 Eliminación de Libro



Diálogo de confirmación de eliminación

Al hacer clic en "Eliminar" se muestra un diálogo de confirmación. Si el usuario confirma, se ejecuta `DELETE /libros/:id` y aparece un toast de éxito.

4.6 Notificaciones Toast



Toast de éxito

Los toast aparecen durante 3 segundos en la esquina de la pantalla y se pueden cerrar manualmente.

5. Conclusiones

Logros obtenidos

El proyecto cumple con todos los requisitos obligatorios del trabajo práctico y añade

funcionalidades extra que mejoran la experiencia de uso:

- La separación en capas (Frontend / Backend / Datos) permite modificar cada parte de forma independiente sin afectar al resto.
- Las validaciones se implementaron en ambas capas: Zod en el frontend evita enviar datos inválidos al servidor; el middleware `validateLibro` en el backend garantiza la integridad aunque se acceda directamente a la API.
- La autenticación por API Key agrega una capa de seguridad básica que impide el acceso no autorizado a los endpoints.
- La persistencia en PostgreSQL mediante Sequelize permite que los datos sobrevivan entre reinicios del servidor y deploys en la nube (Render), garantizando integridad real de los datos.

Dificultades encontradas

Sincronización del estado entre frontend y backend: al agregar o editar un libro, fue

necesario volver a hacer un `GET /libros` para refrescar la tabla, en lugar de actualizar el estado local directamente. Esto se resolvió dentro del hook `useBooks` llamando a `fetchBooks()` luego de cada operación mutante (POST, PUT, DELETE).

Tipado de datos de formulario con Zod y react-hook-form: el campo `anio` proviene del input HTML como string pero la API espera un número. Se resolvió con la opción `{ valueAsNumber: true }` en `register()`, y validando el tipo `number` en el esquema Zod.

Manejo de errores de Axios: los errores HTTP (400, 401, 404) no llegan como excepciones estándar de JavaScript. Se implementó la función `extraerMensajeError` que usa `axios.isAxiosError` para extraer el campo `error` del body de respuesta y mostrarlo en el toast correspondiente.

CORS en deploy: en desarrollo, frontend (puerto 5173) y backend (puerto 3001) corren en la misma máquina. En producción, el frontend está en Vercel y el backend en Render (dominios distintos), por lo que el navegador bloquea las peticiones por defecto. Se resolvió configurando el middleware `cors` con el origen exacto del frontend vía la variable de entorno `CORS_ORIGIN`, que en producción se setea con la URL de Vercel.